

IT2214 Predictive Analysis and Forecasting – Assignment Report

Predicting Calories Burned from Gym Members' Exercise Data

Name: Ginja Hasini

Admin Number: 250540C

Tools used:

KNIME: Data Preparation & Exploration

SAS Viya: Modelling & Evaluation, full exported output of all 16 models and 3 comparisons:

<https://it2214-paf-hasini.netlify.app/charts.pdf>

Overview

From 4,865 raw records I built a clean 950-member dataset in KNIME (3,892 duplicates and 23 physiologically impossible rows removed, zero missing values, BMI independently validated). Subsequently, I compared sixteen model variants across six families in SAS Viya, all on one fixed 70/30 partition. The best model is a 13-predictor Gradient Boosting model predicting Calories_Burned to within **±18.7 calories** (validation ASE 349.61, 99.53% of variance removed vs the mean only null). I picked it on the validation partition after noticing SAS Viya selects its champion on training ASE and its rank held on an independent 80/20 re-split. The most unexpected finding is that members **burn approximately 718 calories per hour, regardless of experience level, workout type, or body composition. Only physiologically determined rates differ (intensity, Gender, Age) that no program can change.** Total burn is essentially rate × time, so how long someone trains matters more than what they do. Recommendations build on this.

1. Introduction & Analytical Approach

The operator wants to understand which factors drive Calories_Burned per session and to have models that quantify those effects. The target is continuous, so this is a regression problem. I ruled out forecasting early, as the data are cross-sectional with no time axis. As per the brief, cleaning and exploration were done in KNIME, modelling and evaluation in SAS Viya. One thread runs through the report: Session_Duration_hr is the dominant driver ($r = 0.91$), all sixteen variants rank it first, and the recommendations rest on a burn-rate mechanism (≈ 718 cal/hr).

2. Data Cleaning & Transformation (KNIME)

Figure 1 shows the pipeline. I added two detect-only side branches (a missing-value check and a numeric outlier check) that inspect the data without touching the modelling stream. I only deleted rows where I could point to a concrete rule, and both engineered features have a physiological basis.

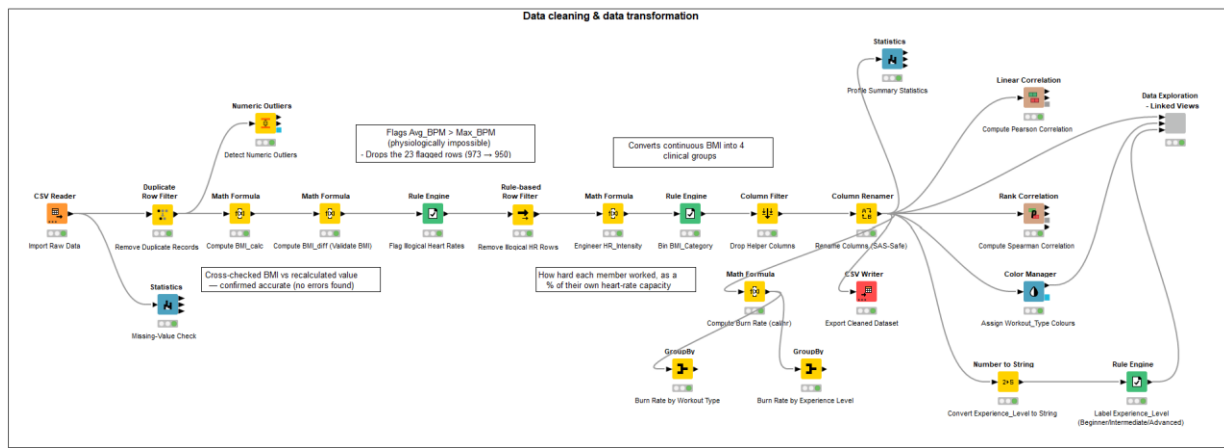


Figure 1. End-to-end KNIME pipeline: cleaning, validation, feature engineering, export, exploration branch and burn-rate nodes.

Step	Rule & justification	Rows
Import	4,865 raw records read (CSV Reader)	4,865
Deduplicate	3,892 exact duplicates removed — 80% of the file was replication, which would inflate statistics and leak identical rows across the train/validation split	973
Missing / outlier audit	Zero missing values, verified not assumed. Numeric Outliers ($IQR \times 1.5$, detect-only), flagged values are physiologically plausible, so retained. Removing them would drop the high-value members most worth understanding	973
BMI validation	Recomputed $Weight/Height^2$; max discrepancy 0.005, so recorded BMI is confirmed accurate. A validation, not a transformation.	973
Illogical heart rates	$Avg_BPM > Max_BPM$ is physiologically impossible (an average cannot exceed its own maximum). Rule verified against the data, 23 rows dropped	950
Feature engineering	$HR_Intensity = (Avg - Rest) / (Max - Rest)$ BPM — the Karvonen heart-rate reserve (Karvonen et al., 1957). BMI_Category — the four WHO clinical bands (WHO, 2000)	950
Export	Helper columns dropped, SAS-safe names and cleaned dataset written	950×17

Both engineered features are later parsimony-tested and the champion leaves them out. I did not assume they would help. I tested it, and working out why they turned out redundant became a useful evaluation finding.

3. Data Exploration (KNIME)

All exploration charts branch off the same cleaned output the models use, so every chart describes the training data exactly. The exploration views are bundled into a KNIME Component with linked selection on. Target: Calories_Burned spans 303–1,783, mean ≈ 905 , skewness $+0.29$ (Statistics node), a mild right skew from a tail of long sessions. That is why I test a Gamma/log GLM later, though the skew is too mild to justify log-transforming every model.

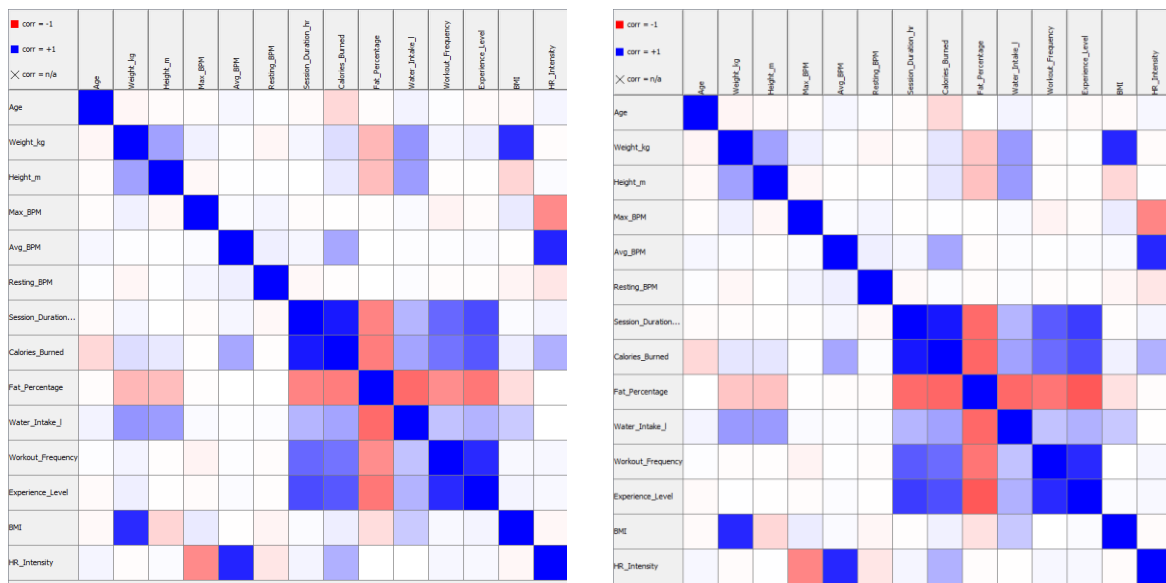


Figure 2. *Pearson (a) and Spearman (b) agree on the predictor ranking — the driver hierarchy is not an artefact of linearity assumptions*

I ran both correlation nodes on purpose: Pearson captures linear association, Spearman any monotonic one, so disagreement would mean the ranking could not be trusted. Reading the coefficients from the two nodes' output tables rather than the heatmap colours, they agree (Pearson / Spearman): Session_Duration_hr $+0.909$ / $+0.906$, Experience_Level $+0.696$ / $+0.661$, Fat_Percentage -0.601 / -0.510 , Workout_Frequency $+0.581$ / $+0.549$, Water_Intake_l $+0.366$ / $+0.356$, Avg_BPM $+0.350$ / $+0.345$, HR_Intensity $+0.307$ / $+0.308$, Weight, Height, BMI, Max_BPM and Resting_BPM near zero in both (the basis of R5). Duration's dominance is not an artefact of linearity. Two details matter later. Fat_Percentage loses most under Spearman, an early sign of the duration confound R4 quantifies, and Avg_BPM and Age (-0.154) rank second and third in the champion (Figure 8) despite weak marginal correlations.

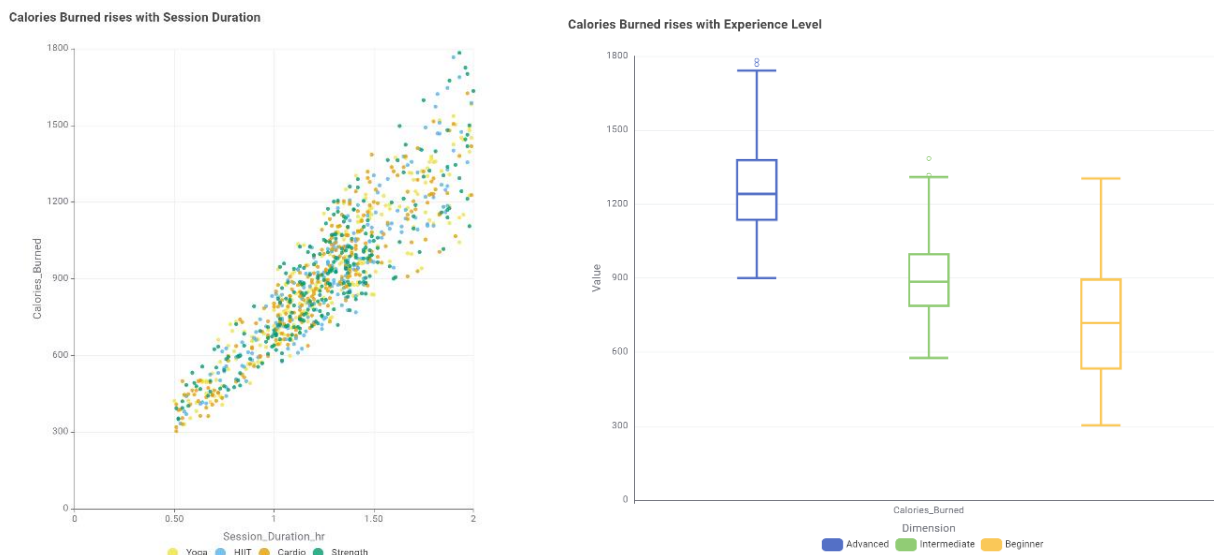


Figure 3. *(a) One tight rising band, all four workout types intermingled. (b) The experience staircase — total burn rises $\approx 74\%$ from Beginner to Advanced, which the modelling and recommendations show is a duration effect.*

Three questions came out of these charts, with the supporting figures drawn from the GroupBy aggregation branch of the workflow rather than the charts themselves. First, the duration scatter is one tight band with all four workout types mixed in (mean burn spans only Cardio 886 to HIIT 919, 0.12 SD), so modality looks irrelevant. Second, the experience staircase looks like efficiency but is really a duration effect (R0/R3): advanced members train 1.76 h versus beginners' 1.01 h at the same ≈ 718 cal/hr. Third, the fat-percentage relationship ($r = -0.601$) is negative but mostly a session-length confound (R4). The modelling then answered all three.

4. Modelling (SAS Viya)

4.1 Test design: held constant across every model

Everything below shares one setup. **Partition:** a single 70/30 split (665 train / 285 validation), fixed seed, reused by every model. **Response:** Calories_Burned (interval). **Predictors:** 16 (13 continuous plus Gender, Workout_Type, BMI_Category; Experience_Level_Cat dropped as a duplicate encoding). **Selection:** validation ASE throughout, never training ASE, because held-out error is the only honest estimate of performance on unseen members. **Left out:** Logistic Regression and SVM (categorical targets only) and Forecasting (no time axis). Six families grew into **16 documented variants over 38+ recorded runs** through tuning, diagnostics and parsimony experiments.

Why six families, not one. Each family was included to test a different assumption, not to pad a leaderboard. Linear Regression and the GLMs ask whether the relationship is essentially additive and what error distribution it follows. A single Decision Tree asks whether hard splits capture the structure and gives an interpretable floor. Forest asks whether averaging independent trees beats one tree. Gradient Boosting asks whether sequential residual fitting beats averaging. The Neural Network asks whether any non-linear structure remains that none of the others can reach. Running all six meant the champion was chosen against real alternatives, and the families that lost were informative in their own right: the GLM sequence isolated the cost of the log link and the Gamma distribution, and the Neural Network's failure quantified how little non-linear structure survives once duration is in the model.

Auto-stop method:
None

Number of trees:
100

Learning rate:
0.1

Subsample rate:
0.5

Maximum branches:
2

Maximum levels:
5

Leaf size:
5

Predictor bins:
100

Figure 4. The tuned Gradient Boosting configuration in Explore & Visualize — 100 trees, learning rate 0.1, subsample 0.5, maximum branches 2, maximum levels 5, leaf size 5, 100 predictor bins. Auto-stop is off, so all 100 trees are grown.

4.2 Results: all sixteen variants (both partitions)

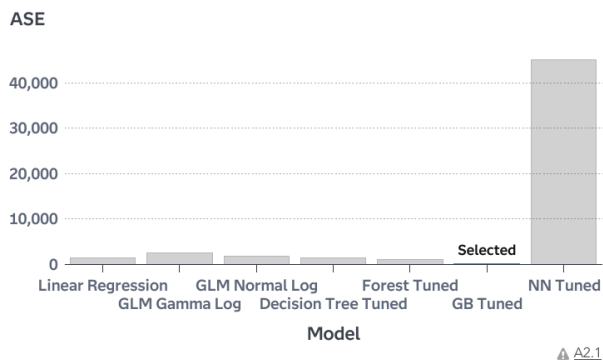
Model variant	Key configuration	70/30	80/20
GB Reduced — CHAMPION	13 predictors (BMI, BMI_Category, HR_Intensity removed); 100 trees, LR 0.1, subsample 0.5, max levels 5, leaf 5, 100 bins	349.61	266
GB Tuned	16 predictors; same tuned settings	393.86	318
GB Autotune	SAS genetic-algorithm search, 50 models	479	≈340
Gradient Boosting (default)	50 trees, LR 0.1, subsample 0.5	605	≈470
Linear Regression	Backward α 0.01 → Duration, Avg_BPM, Gender, Age; R^2 0.9806	≈1,700	≈1,900
LR (Interaction)	+ Duration × Experience — null result	≈1,700	≈1,900
GLM Normal + Log	Diagnostic isolation run	≈2,000	≈2,100
Forest Tuned	100 trees, bootstrap 0.8, vars/split 10, levels 12, leaf 5	2,605.02	≈2,100
Forest Reduced	13 predictors, vars/split 8 (controlled)	2,674.39	≈2,000
GLM Gamma + Log	Distributionally principled variant	≈2,800	≈3,400
Decision Tree Tuned	2 branches, levels 10, leaf 5, 100 bins, validation pruning	≈5,100	≈5,200
Forest (default)	100 trees, bootstrap 0.6, $\sqrt{p}$ vars/split	≈7,600	≈7,500
Decision Tree (default)	SAS defaults	≈8,200	≈6,800
GB Baseline (Duration only)	Single predictor — benchmark	≈16,000	≈15,000

Model variant	Key configuration	70/30	80/20
NN Tuned	2 layers × 10, Tanh, Midrange, 25 iterations	≈40,000	≈73,000
NN (default)	1 × 10; stopped at 4 iterations — mean-predictor	≈71,000	fail

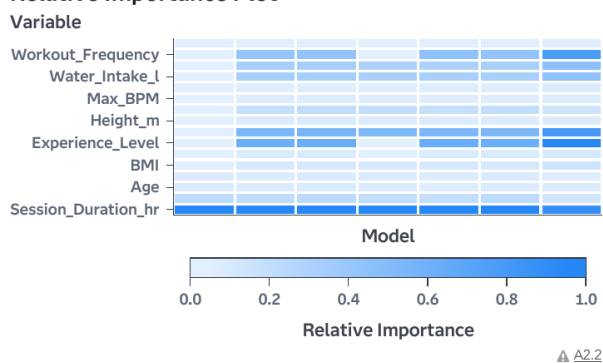
Three notes on the Linear Regression result. First, backward selection threw out Experience_Level and Fat_Percentage even though their raw correlations are +0.696 and −0.601. The reason is multicollinearity: Session_Duration_hr absorbs their shared variance, so once duration is in the model they add nothing independent. Second, and more interesting, is what selection **kept**: just four predictors: Session_Duration_hr, Avg_BPM, Gender and Age. It took me a burn-rate decomposition to see why. Because total burn is rate × time, a predictor can enter only by moving one of the two. Duration is the time term. The other three are **rate** terms: Avg_BPM correlates **+0.806 with cal/hr** (against only +0.350 with total calories), and Gender and Age each shift the hourly rate by about 10% at effectively identical session lengths — males 750.7 vs females 681.6 cal/hr, both averaging 1.26 h, and the youngest age quartile 752.1 vs the oldest 676.9, on session lengths of 1.23–1.30 h. That also resolves a puzzle in the importance plots: Age ranks third in the champion despite correlating only −0.154 with total calories, because its effect is on the rate, not on the total. So the four survivors are not an arbitrary set. They are one time variable and three rate variables. **Third, metric choice is not driving the ranking:** ASE, R² and the information criteria (AIC 5,526.7, SBC 4,882.2, Adj R² 0.9805) all order the likelihood-based models the same way. AIC/SBC do not exist for the tree ensembles, which have no likelihood function, which is why those cells are blank.

Model Comparison of Calories_Burned

Fit Statistic



Relative Importance Plot



Assessment

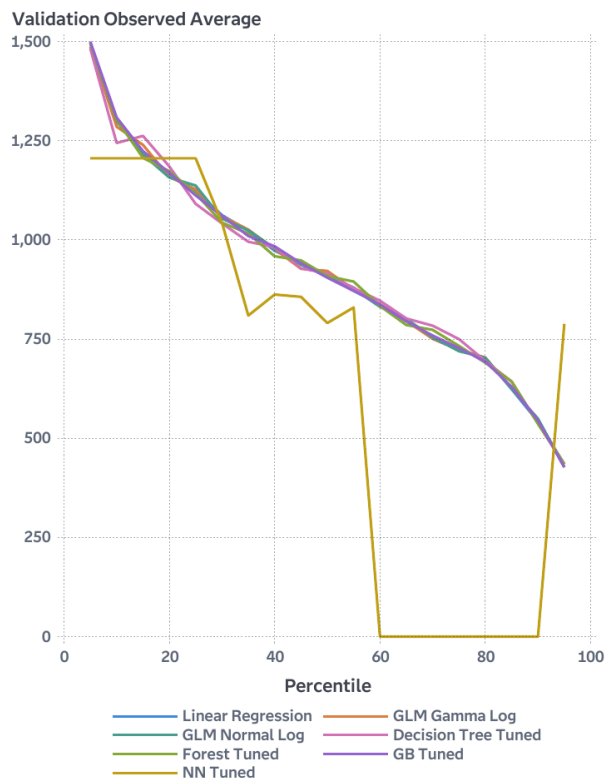


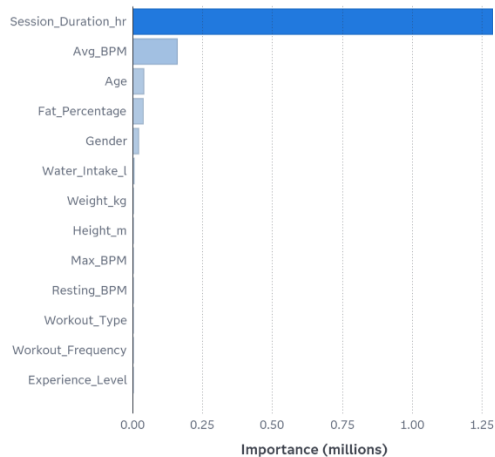
Figure 5. Comparison 1 — the seven primary models with roles, response and partition held constant. Three things read at once: the Neural Network's ASE towers over every other family; SAS's automatic "Selected" label lands on GB Tuned (the training-ASE catch, section 5.2); and in the Assessment panel every model tracks the observed average except NN Tuned, which collapses between the 60th and 90th percentiles. The champion appears in detail in Figure 8.

Partition stability across all sixteen variants. I re-fitted every variant on an independent 80/20 split; Figure 6 shows the champion on that split. The champion held its rank, and a clear pattern emerged: complexity and partition sensitivity track together. The one-variable baseline barely moved (−6%); the four Gradient Boosting variants moved together (−19% to −29%), keeping their order (Reduced < Tuned < Autotune < default); and the Neural Network, most parameters, least data per parameter, swung +83%, an order of magnitude beyond any other family. The tuning gain held almost identical on both splits (−42% vs −43%), so it captured real structure, not quirks of one partition. Untuned Linear Regression, with nothing to overfit, moved 12% (≈200 ASE), the noise floor I used to judge which model differences are real.

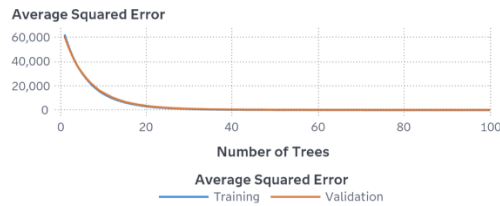
Gradient Boosting of Calories_Burned

Fit: Validation ASE 266 ▾ Observations: 950 of 950

Variable Importance



Iteration Plot



Assessment



Figure 6. The champion re-fitted on the independent 80/20 split — validation ASE 266. The importance list carries its own identification: exactly 13 predictors, with BMI, BMI_Category and HR_Intensity absent, and the same Session_Duration_hr › Avg_BPM › Age ordering as the 70/30 fit.

4.3 Hyperparameter tuning – every parameter bracketed

I swept each tunable family one hyperparameter at a time (Forest 13 runs, Decision Tree 10, Gradient Boosting 9, Neural Network 7), keeping the runs that got worse as well as those that helped and bracketing each optimum on both sides, so it is a genuine interior optimum rather than the edge of what I tried. The decisive runs are below; further confirmatory runs are omitted for space but followed the same design.

Family	Default → Tuned	Gain	What the sweep found (mechanism)
Gradient Boosting	605 → 393.86 (→ 349.61 reduced)	35% (42%)	100 trees at LR 0.1 converged fully; bins 50→100 helped, 150 hurt (interior optimum); shallow trees won — deeper trees (depth 8) worsened ASE to 689
Forest	7,563 → 2,605.02	66%	Raising vars/split from the $\sqrt{16} \approx 4$ default was the single biggest lever (–42% alone): with one dominant predictor, $\sqrt{p}$ sampling excluded Session_Duration_hr from $\approx \frac{1}{4}$ of all splits. Depth 6→12 and bootstrap 0.8 added more
Decision Tree	8,202 → $\approx 5,100$	38%	Depth turns at 10 (12 is worse) with validation pruning and 100 bins; leaf size 5 guards against memorising noise
Neural Network	71,000 → $\approx 40,000$	44%	Diagnostic sweep — improved but never competitive (below)

Why depth reverses between bagging and boosting. The strangest result in the sweep: deeper trees helped the Forest (6→12) but made Gradient Boosting worse (479→689 at depth 8). Same data, same hyperparameter, opposite direction. A Forest grows trees independently and averages them, so each needs depth to capture structure on its own, and the averaging cancels the extra variance. Boosting is sequential, the depth comes from the chain of trees, not any single one, so deep individual trees overshoot into noise later trees must undo. Binning had an interior optimum in both tree models too: 50→100 bins placed splits on Session_Duration_hr more precisely (DT 5,400→5,100; GB 479→394), but 150 began to chase noise.

How I tuned the Neural Network (and why it still failed). This is the one family I could not rescue, and the sweep is worth showing because the way it failed is itself the finding. The default (one hidden layer of 10 neurons) gave up after just 4 iterations and predicted the mean for every member — a dead flat line in the Assessment plot at ASE $\approx 71K$. My first hypothesis was that it was stopping too early, so I raised the iteration limit to 50, then 100 (Runs A and B). Both stayed stuck at the same 4 iterations and the same 71K, which killed the early-stopping idea; the optimiser was settling into a flat solution, not being cut short. That pointed at capacity, not training time. Run C widened the single layer to 20 neurons; it trained longer but could not compute an assessment, so it was unusable. Run D added a second hidden layer (10 + 10 neurons, Tanh activation, Midrange standardization): this finally let the network train a full 25 iterations and reach $\approx 40K$, a 44% improvement and the best I got. Run E tried standard-deviation standardization on top of D, which broke the assessment again and was rejected. SAS's own export backs this up: the appendix logs "The optimization exited on maximum iterations" against every NN Tuned object.

Run	Configuration	Val ASE	Outcome
Default	1 × 10 neurons, default iterations	≈71K	stopped at 4 iterations — predicts the mean
A / B	1 × 10, max iterations 50 / 100	≈71K	still 4 iterations — early-stopping hypothesis falsified
C	1 × 20, max iterations 100	assessment failed	trained but could not compute assessment
D	2 × (10 + 10), Tanh, Midrange, 25 iter	≈40K	best — 44% improvement
E	D + standard-deviation standardization	assessment failed	rejected

Even at its best the network trails every tree model. With 665 training rows and a target that is near-linear in one dominant predictor, a high-capacity approximator has too many parameters per observation and no non-linear structure left to exploit that simpler models miss. It was also the least stable, moving +83% across partitions.

Interior optima, not edge effects. A tuned value only means something if a neighbouring run was worse. The table records, for each parameter, the value I chose and the neighbouring value or values I tested, two-sided where the parameter allowed it (Gradient Boosting bins, Neural Network iterations), one-sided where the other direction was the default I had already beaten. It is the evidence that these are interior optima rather than the edge of what I happened to try.

Family	Parameter	Chosen	Also tried	What the neighbouring run showed
Gradient Boosting	Predictor bins	100	50 / 150	50 gave 479; 150 began chasing noise — optimum is interior
Gradient Boosting	Maximum levels	5	8	depth 8 worsened validation ASE to 689
Forest	Variables per split	10	√16 ≈ 4 (default)	the single biggest lever: vp sampling excluded duration from ≈¼ of splits
Forest	Maximum levels	12	6	deeper helped here — the opposite of Gradient Boosting
Decision Tree	Maximum levels	10	12	12 was worse; the turn is at 10
Decision Tree	Predictor bins	100	50	5,400 → ≈5,100; finer splits on duration
Neural Network	Iteration limit (1 × 10 net)	default	50 / 100	raising the cap changed nothing: still 4 iterations, still ≈71K
Neural Network	Hidden layers	2 × 10	1 × 10	one layer sat at ≈71K; two reached ≈40K

4.4 Controlled experiments beyond tuning

- **GLM link/distribution decomposition:** I fitted three GLM configurations with identical roles so each change isolates one assumption — the Normal/Identity case is the Linear Regression object itself, which is mathematically a GLM with identity link, so the demo shows two dedicated GLM objects plus the regression. Normal/Identity (≈1.7K) to Normal/Log (≈2.0K) isolates the log link, costing ≈0.3K; Normal/Log to Gamma/Log (≈2.8K) isolates the distribution, costing another ≈0.8K. So the distributionally “correct” Gamma actually loses. It took me a while to see why: the skew lives in the marginal distribution of the target, but the conditional relationship (calories given duration and heart rate) is close to linear with fairly constant error, so once the predictors are in there is little skew left for a Gamma to fix. It paid for a constraint the data did not need.
- **Interaction null:** A Duration × Experience_Level interaction came back null three separate ways: $p = 0.828$, a drop in adjusted R^2 , and it was the very first term backward selection eliminated. The same null from three independent methods is what lets R3 stand.
- **Parsimony (both ensembles):** Removing the three derived features improved Gradient Boosting 11% (393.86→349.61) but left the Forest unchanged (2,605.02 vs 2,674.39, a 69.37 gap inside the noise floor). Boosting fits residuals in sequence, so redundant correlated features give later trees extra chances to chase noise, bagging averages independent trees and does not care. I should also admit my first Forest Reduced run was confounded: I left vars/split at 10 while predictors fell 16→13, which changed the proportion sampled per split. I re-ran at the controlled 8 and report only that.
- **Manual tuning vs autotune:** SAS’s autotune (50 models) and my manual tuning first landed on the exact same 479, which was reassuring, because two independent searches agreeing suggests it is a real optimum. Manual tuning then pulled ahead to 393.86 by carrying a hypothesis across models: finer binning had helped the Decision Tree, so I tried it on Gradient Boosting and it worked too. The autotune never tried that, because bins sat outside its search space. Automated search gives you a floor, but it does not think across models.

5. Model Evaluation (SAS Viya)

5.1 Three comparison objects with stated controls

I set up three Model Comparison objects rather than one leaderboard, so each is a controlled comparison. **Comparison 1** holds roles, response and partition constant across the seven primary models (best per family). **Comparison 2 (parsimony)** holds response and partition constant and varies only the predictor set (GB Tuned, GB Reduced, Forest Tuned, Forest Reduced, GB Baseline). **Comparison 3** is the readable ranking: it removes the Neural Network and the Gamma GLM (whose errors compress every other bar to nothing), adds GB Reduced, and ranks on validation ASE; there the 'Selected' label lands on GB Reduced. Figure 5 is Comparison 1, where the label lands on GB Tuned. I come back to that selection in the next subsection.

5.2 Methodological catch: SAS selects its champions on training ASE

Champion Model Summary

The champion model is GB Tuned. The model was chosen based on ASE (86.09) for the training partition. The 8 most important factors with greater than 10% relative importance are Session_Duration_hr, Experience_Level, Fat_Percentage, Workout_Frequency, Water_Intake_L, Weight_kg, Avg_BPM, and HR_Intensity.

Figure 7. SAS Viya's exported Champion Model Summary — the automatic label names GB Tuned on training ASE (86.09). On validation the ordering reverses, so I overrode it.

The exported Champion Model Summary says the champion "was chosen based on ASE (86.09) for the training partition". That 86.09 is GB Tuned's **training** error, so SAS is picking its champion on the training set. When I checked the validation partition myself the ordering flipped and GB Reduced won, so I overrode the automatic label and chose on validation ASE. Underneath, this is the bias-variance trade-off in my own numbers, and the generalisation gap grows steadily with model capacity:

Model	Inputs	Train ASE	Validation ASE	Val/Train
GB Baseline	1	≈11,000	≈16,000	≈1.45
Forest Reduced / Tuned	13 / 16	1,219/≈1,200	2,674.39 / 2,605.02	2.19/≈2.2
GB Reduced — champion	13	95.44	349.61 (better generalisation)	3.66
GB Tuned	16	86.09 (better fit)	393.86	4.57

GB Reduced fits the training data worse than GB Tuned (95.44 vs 86.09) but generalises better (349.61 vs 393.86): the three derived features gave the 16-variable model extra capacity, and it spent it fitting training noise that did not transfer. The redundancy is easy to demonstrate after removal the importance profile of the remaining 13 predictors barely moved, every value shifting under 0.002 (Experience_Level 0.6265→0.6254, Fat_Percentage 0.5389→0.5379), so the derived variables held nothing their parents did not. To be fair, the champion's own 3.66× gap is still the second largest I measured, a genuine concern and the reason quarterly monitoring is in R6.

5.3 Is this champion actually good? Benchmark ladder & accuracy

Benchmark	Validation ASE	Variance removed vs null
Null model (predict the mean; variance of Calories_Burned across all 950 members)	74,403	—
GB Baseline — Session_Duration_hr only	≈16,000	78.5%
Best non-GB family (Forest Tuned)	2,605.02	96.5%
Champion — GB Reduced	349.61	99.53%

A table of ASE values does not really tell you whether a model is good, so I benchmarked against a null model that predicts the mean. Duration alone already removes 78.5% of validation variance, consistent with its exploration r^2 of 0.83; the champion removes 99.53%, which as a validation RMSE is **±18.7 calories on a mean of ≈905 (2.1%)**, easily accurate enough to show a member at booking time. Put in operational terms, **±18.7 calories is about 1.6 minutes of training** at the ≈718 cal/hr burn rate, against 3.4 minutes for Linear Regression, comfortably inside the precision a member would notice on a booking screen.

5.4 Noise-floor significance test

To tell which gaps are real, I used untuned Linear Regression as a control: with nothing tuned, its ≈200-ASE (12%) partition movement is a noise floor. The champion's ≈ 1,350-ASE lead over it is ≈6.8× the floor (real); the 44-ASE parsimony gain sits inside it, so I claim GB Reduced only as "no worse with three fewer inputs," not more accurate.

5.5 Champion diagnostics, model card, and the case against the champion

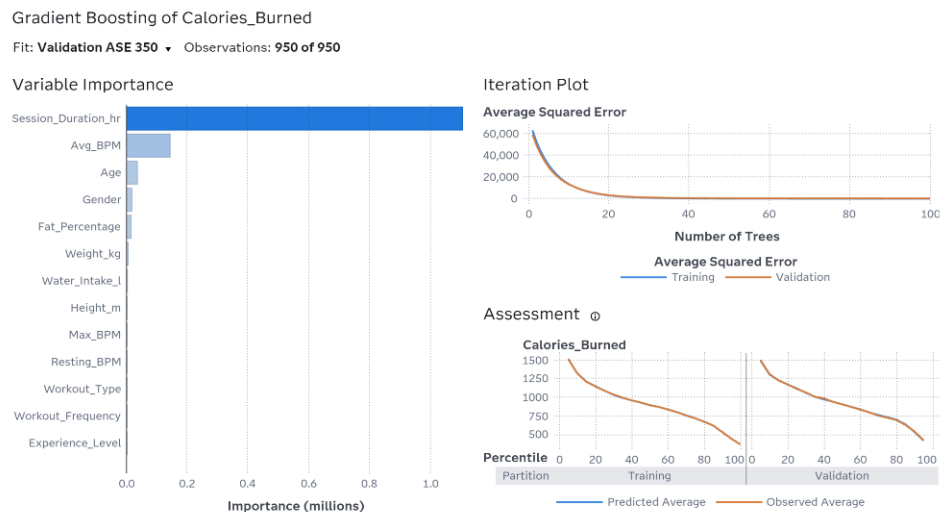


Figure 8. *Champion diagnostics: converged iterations, predicted and observed averages overlapping across the full percentile range — no systematic bias at either tail, on either partition.*

The Assessment plot works like a calibration curve for regression (predicted vs observed by percentile bin). I checked whether the champion under-predicts the top decile or drifts at the low end, and it does neither. The lines track across the whole range on both partitions, so estimates are safe for casual and heavy users alike. A confusion matrix or ROC does not apply to an interval target. The iteration plot shows training and validation error converging with no divergence, so the tree count is not overfitting.

Model card	Specification
Model / config	Gradient Boosting, 13 predictors · 100 trees · LR 0.1 · subsample 0.5 · max levels 5 · leaf 5 · 100 bins
Target / data	Calories_Burned (interval) · 950 members, cross-sectional · 70/30 partition
Val ASE / RMSE	349.61 / ± 18.7 calories (2.1% of ≈ 905 mean)
Excluded inputs	BMI, BMI_Category, HR_Intensity — derived; removal eliminates all scoring-time preprocessing
Intended use / scope	Booking-time calorie estimate · not medical/dietary advice, forecasting or causal inference
Monitoring	Quarterly retrain · drift alert if rolling RMSE $> \pm 30$ calories (1.6 $\times$ current)
Known weaknesses	Hyperparameters inherited from the 16-variable tune · partition sensitivity -19% to -24%

To argue against my own champion for a moment: Gradient Boosting halves Linear Regression's error (± 18.7 vs ± 41 calories) but cannot give you a sentence a staff member could repeat. The regression can: each extra 30 minutes is associated with roughly +349 calories, holding the other measured factors constant. That 349 comes from the controlled multivariate coefficient (698 cal/hr), not the confounded raw scatter slope of 723. My actual recommendation is to deploy both: GB Reduced does the scoring, and Linear Regression stays on as the explainable model for staff and marketing.

6. Recommendations

Each recommendation follows finding $\rightarrow$ evidence $\rightarrow$ action $\rightarrow$ impact. Burn rate ($\text{Calories_Burned} \div \text{Session_Duration_hr}$) was computed from the cleaned data for description only; it was never a model input and is not in the exported 17-column modelling dataset, so there is no target leakage.

Segment	Burn rate (cal/hr)	Spread
Experience: Beginner / Intermediate / Advanced	715.1 / 720.1 / 718.6	0.7%
Type: Cardio / HIIT / Strength / Yoga	721.7 / 713.6 / 722.0 / 713.3	1.2%
Body-fat quartile Q1 $\rightarrow$ Q4	726 / 735 / 717 / 692	4.7%
Gender: Female / Male (rate effect)	681.6 / 750.7	10.1%
Age quartile Q1 $\rightarrow$ Q4 (rate effect)	752 / 746 / 697 / 677	10.0%
Overall	718 $\pm$ 86	—

- **R0 — calorie burn is a near-constant rate × time process (master finding).** Across experience, workout type and body composition, calories per hour never varies more than about 5% around 718. Total burn is driven almost entirely by **how long members stay**, not who they are or what class they pick. The exceptions are physiological, not behavioural: intensity (Avg_BPM) correlates +0.806 with cal/hr, and Gender and Age each move the rate by ≈10% at identical session lengths. Nothing the operator programmes changes the rate; only how long members stay. This explains R1–R4 and changes the member-facing question from “what should I do?” to “how long can you stay?”
- **R1 — session duration is the primary commercial lever.** $r = 0.91$, #1 importance in all tree models, root split, 78.5% of variance alone. **Action:** show projected burn at booking, promote 90-min over 60-min slots. **Impact:** bookings ≥1.5 h average 1,260 cal vs 546 for ≤1 h; each +30 min ≈ +349 cal; 19.8% of sessions are ≤1 h — the largest addressable segment.
- **R2 — workout type does not matter for calories.** means Cardio 886 / Yoga 905 / Strength 911 / HIIT 919 — a 33-cal spread (0.12 SD), eliminated at step 4 ($p = 0.598$), near-zero importance. **Action:** stop marketing any class as “best for calorie burn”; market duration and adherence. **Impact:** members choose the modality they will keep attending — adherence, not modality, is the lever.
- **R3 — experience works through duration, not alongside it.** experience correlates +0.70 with total burn, yet burn rate is 715/720/719 across levels (0.7%); advanced members simply train 1.76 h vs 1.01 h. Confirmed non-significant in regression, eliminated at step 5, interaction null under three tests. **Action:** target duration progression at every level, not tiered “efficiency” programmes.
- **R4 — use body composition to set realistic expectations.** raw $r = -0.60$, but burn rate falls only 726 → 692 cal/hr across fat quartiles (−4.7%). **Action:** personalise targets — a member at 32% body fat should expect ≈5% lower hourly burn than one at 16%, not the 33% implied by raw totals.
- **R5 — simplify data collection; drop the derived-feature pipeline.** Max_BPM, Resting_BPM, Height, Weight near-zero everywhere (Max_BPM eliminated at step 7); parsimony improved the champion 11% with three fewer inputs. **Action:** score on raw fields only; retire derived-feature preprocessing — no preprocessing at scoring time, fewer fields, marginally better predictions.
- **R6 — deployment plan.** Deploy 13-variable GB Reduced as the booking-time estimator; retain Linear Regression as the explainer. Retrain quarterly; drift alert if rolling RMSE exceeds ±30 cal (1.6× current); re-tune on the 13-variable set at first retrain, closing the one known specification weakness.

7. Limitations

- **Causality & time:** cross-sectional data supports association, not causation, and has no time axis, so no forecasting is possible.
- **Sample & partitioning:** 950 usable rows; ensemble results move 19–29% across partitions at this size. Explore & Visualize permits one partition indicator, so stability was assessed by a manual 80/20 re-split (Figure 6).
- **Structural optimism:** hyperparameters were tuned on validation ASE and the champion’s R^2 of 0.98 reflects a near-deterministic target, so this accuracy should not be assumed to hold on independently measured operational data (optimism quantified via the 80/20 re-split; champion settings tuned on 16 variables, inherited by 13, addressed in R6).
- **No cost/revenue data:** impacts are in calories; commercial value (retention, tier uptake) needs financial data absent here.

8. Conclusion

Three things carried this analysis. Cleaning removed 3,892 duplicate records (80% of the raw file) and 23 physiologically impossible rows, leaving 950 members whose statistics can be trusted. Sixteen variants across six families, all on one fixed test design, produced a 13-predictor Gradient Boosting champion accurate to **±18.7 calories**; I selected it on the validation partition after finding that SAS Viya names its champion on training ASE, and its rank held on an independent 80/20 re-split. And exploration, modelling and the burn-rate analysis converged on one mechanism from three directions: members burn **≈718 calories an hour** whoever they are and whatever they train, so total burn is rate × time and **session duration is the only lever with real leverage**. That mechanism, not the model itself, is the finding the operator can act on, and R0–R6 turn it into concrete changes to booking, marketing, member targets, the data pipeline and a monitored deployment.